

CSC347 A1 — Security Vulnerability Report

Reconstructed from the assignment notes provided in chat. This is a reconstructed report, not a recovered copy of the original submitted document. Fix status below reflects the notes supplied.

Important build and testing notes

- Compile with: `cc account.c -o account -lcrypto`. The `-lcrypto` flag is required for the OpenSSL MD5 hashing routine described under Issue 7.
- Run `./setup_bash.sh` to prepare the test environment, then compile the program.
- The notes refer to a separate Issue 3 test/exploit file at `exploits/accounts_issue3.c`. Preserve and include that file if it is part of the assignment package.
- For convenient testing of Issue 6, the notes say to comment out the `getenv("USER")` line around line 317 and use `getName()` instead. The secure version should use `getName()`; `getenv("USER")` is only a test convenience.
- Issue numbers in the original notes refer to line numbers in the unchanged `accounts.c`. In the fixed version, comments such as `// ISSUE NUMBER*****` identify corresponding code changes.

Issue 1 — Unterminated username copy

Fix status in notes: YES

Vulnerability: Vulnerability: The username is copied using `strncpy(user, getenv("USER"), 100)`. If `USER` is at least 100 bytes, the destination may not be null-terminated. Later string operations can read beyond the intended username, corrupt account naming, and interfere with authentication.

Exploit / example: Example test: `USER=$(perl -e 'print "A"x100'); ./account rip`. The notes report that a 100-character username can be written but the same password may then fail to authenticate because the username buffer is not properly terminated.

Impact: Impact: Account ownership and authentication data may become corrupted, affecting integrity and accountability.

Fix / recommendation: Fix described: Obtain the name, provide a fallback if unavailable, copy at most `sizeof(user)-1` bytes, and explicitly terminate `user[sizeof(user)-1] = '\0'`.

Issue 2 — Path traversal when writing account files

Fix status in notes: YES

Vulnerability: Vulnerability: `setAccount()` builds an account path from the username. A username such as `../passwords` can escape the accounts directory and overwrite another file.

Exploit / example: Example test: `USER=../passwords; ./account password`. The notes describe the passwords file being overwritten.

Impact: Impact: Unauthorized modification of credential files; integrity is affected.

Fix / recommendation: Fix described: Validate every username character, allowing only alphanumeric characters, underscore, or hyphen, and build the path with `sprintf(fileName, 100, "/vulnerable/accounts/%s", user)`.

Issue 3 — Path traversal when reading account files

Fix status in notes: YES

Vulnerability: Vulnerability: `getAccount()` reads an account file using a username-derived path. Without validation, `../passwords` can cause the program to read a file outside the accounts directory.

Exploit / example: Example test: The notes refer to a separate file, `exploits/accounts_issue3.c`, which demonstrates reading the passwords file without the write-overwrite behavior from Issue 2. It describes setting `USER=../passwords` and observing password-file contents in output.

Impact: Impact: Disclosure of sensitive information, affecting confidentiality and potentially authentication.

Fix / recommendation: Fix described: Apply the same username allow-list validation used for Issue 2 before constructing or opening the account path. Keep the separate Issue 3 test file with the project if required.

Issue 4 — Unsafe transaction construction and logging passwords

Fix status in notes: YES

Vulnerability: Vulnerability: `main()` concatenates transaction data without adequate bounds checks. Passwords and attacker-controlled strings may be written to the log, and long input may overflow the transaction buffer.

Exploit / example: Example test: `./account passwd "10 cd ../" root`. The notes show a log entry containing the password and extra user-supplied text.

Impact: Impact: Password exposure in logs, malformed audit records, and potential buffer overflow.

Fix / recommendation: Fix described: Initialize the transaction with `snprintf(transaction, sizeof(transaction), "%s: ", user)`. For a transfer, copy only digit characters from the amount argument into a bounded temporary buffer, terminate it, and append the amount and recipient using `snprintf` with the remaining buffer size.

Issue 5 — Password truncation and inconsistent password handling

Fix status in notes: YES

Vulnerability: Vulnerability: Passwords are effectively capped/truncated at a fixed buffer size. Distinct long passwords sharing the same prefix may be treated as equivalent, while inconsistent length handling can cause authentication problems.

Exploit / example: Example test: The notes use very long passwords generated with Perl and describe successful operations despite different lengths.

Impact: Impact: Authentication integrity is weakened by truncation and inconsistent comparison.

Fix / recommendation: Fix described: Define `MAX_PASS` as 64 and reject passwords whose length exceeds that limit. The notes also describe copying a password into a fixed-size buffer and explicitly null-terminating it as additional defensive handling.

Issue 6 — Trusting the USER environment variable

Fix status in notes: YES

Vulnerability: Vulnerability: The program uses `getenv("USER")` to decide which user is running it. A caller can set this environment variable and impersonate another account.

Exploit / example: Example test: Create accounts using USER=abc and USER=dodo, then change USER back to abc to demonstrate that the identity is controlled by the environment.

Impact: Impact: User impersonation and unauthorized access to or deletion of account data, affecting integrity.

Fix / recommendation: Fix described: Use `getpwuid(getuid())` to obtain the username from the system account database. Copy it safely into a static buffer and null-terminate it; use "invalid" if lookup fails. For convenient testing only, the notes say to comment out the `getenv("USER")` line and enable `getName()`.

Issue 7 — Plaintext password storage

Fix status in notes: YES

Vulnerability: Vulnerability: Passwords are stored as plaintext in `/vulnerable/passwords` and compared as plaintext during authentication.

Exploit / example: Example test: `./account passwd` followed by `cat passwords` reveals the stored password in the vulnerable version.

Impact: Impact: Anyone who can read the password file can learn users' passwords and potentially impersonate them, affecting confidentiality.

Fix / recommendation: Fix described: Add an `md5_hex(const char *str)` helper using OpenSSL and compare the computed hash against the stored value. Compile with `cc account.c -o account -lcrypto`. The notes show a stored MD5 hash and state that entering the hash itself does not authenticate as the original password. Note: MD5 is not considered suitable for modern password storage; a password-specific hash such as Argon2id, scrypt, or bcrypt is preferable where allowed by the assignment.

Issue 8 — Missing files and unchecked `fopen()` results

Fix status in notes: YES

Vulnerability: Vulnerability: The program assumes directories and files such as `/vulnerable/accounts`, `/vulnerable/passwords`, and `/vulnerable/log` exist. If `fopen()` fails and the returned pointer is used anyway, the program may crash.

Exploit / example: Example test: Remove the accounts directory and the passwords/log files, then run `./account passwd`; the notes report a segmentation fault in the vulnerable version.

Impact: Impact: Program crashes and denial of service, affecting availability.

Fix / recommendation: Fix described: Check every `fopen()` result. For files intended to be created automatically, attempt a creation mode if the initial open fails; if creation still fails, report the error and return safely. The accounts directory itself must also exist and be handled safely.

Issue 9 — Transfers to oneself

Fix status in notes: YES

Vulnerability: Vulnerability: The transfer logic does not reject cases where the sender and recipient usernames are identical. This may credit the account without a legitimate transfer.

Exploit / example: Example test: `./account passwd 100 root`, when root is the current user. The notes show the balance increasing incorrectly.

Impact: Impact: Users can inflate balances, violating accounting integrity.

Fix / recommendation: Fix described: Before performing a transfer, compare argv[3] with user. If they match, print "cannot transfer to yourself" and do not perform the transfer.

Issue 10 — Unvalidated transfer amounts and integer overflow

Fix status in notes: YES

Vulnerability: Vulnerability: atoi() does not reliably report invalid input or overflow. Negative, zero, non-numeric, or excessively large amounts may be interpreted incorrectly and corrupt balances.

Exploit / example: Example test: ./account n -200 m. The notes show the fixed version responding with "invalid amount".

Impact: Impact: Unauthorized or incorrect balance changes, affecting integrity.

Fix / recommendation: Fix described: Parse the string using a validation helper that accepts digits only, rejects empty, zero, negative, and values greater than INT_MAX, and reports success/failure. Before transferring, verify sufficient sender funds and ensure the recipient balance will not overflow.

Issue 11 — Logging failed transactions as if valid

Fix status in notes: NO

Vulnerability: Vulnerability: logTransaction() records attempts even when authentication or the transaction fails, such as for insufficient funds or invalid input.

Exploit / example: Example test: Attempt a transfer larger than the sender's balance and inspect the log. The notes show the failed transfer appearing in the log.

Impact: Impact: Audit records become misleading and accountability/integrity are reduced.

Fix / recommendation: Suggested fix in notes: Call logTransaction() only inside the branches that confirm successful authentication and a successful transaction. Record failed attempts separately and label them as failures if audit requirements call for it.

Issue 12 — Unlimited password attempts

Fix status in notes: NO

Vulnerability: Vulnerability: authenticate() allows unlimited failed password attempts and has no delay, counter, or lockout mechanism.

Exploit / example: Example test: Repeatedly run the program with different candidate passwords; the notes describe trying a list of guesses.

Impact: Impact: Repeated guessing may eventually compromise confidentiality.

Fix / recommendation: Suggested fix in notes: Add per-user failed-attempt counters and a temporary lockout or delay after a defined number of failures. A production implementation should also consider rate limiting and safe recovery to avoid easy denial of service.

Issue 13 — Time-of-check/time-of-use and symlink race

Fix status in notes: NO

Vulnerability: Vulnerability: setAccount() opens /vulnerable/accounts/<user> with fopen(..., "w") and then changes permissions. If an attacker can replace the target with a symbolic link at the right time, the program may write to an unintended file.

Exploit / example: Example test described in notes: Create a file such as /tmp/attacker_file, place a symlink at /vulnerable/accounts/victim pointing to it, then trigger account creation and inspect the target file.

Impact: Impact: Unauthorized file modification and potential privilege escalation, affecting integrity.

Fix / recommendation: Suggested fix: Use safer file-creation APIs and flags that prevent following symlinks (for example, open() with O_CREAT | O_EXCL and O_NOFOLLOW where supported), set restrictive permissions at creation, verify the opened file with fstat(), and avoid a separate path-based chmod race.

Issue 14 — Username truncation and collisions

Fix status in notes: YES

Vulnerability: Vulnerability: Username comparisons and fixed-size paths can truncate names. Different usernames with the same retained prefix may map to the same account file or compare as equal.

Exploit / example: Example test described in notes: Construct two usernames that differ only beyond the maximum path/buffer length and attempt to create both accounts; the notes show the names colliding after truncation.

Impact: Impact: Account confusion, data corruption, and possible loss of access, affecting integrity and availability.

Fix / recommendation: Fix described: Define MAX_USER as 64 and reject usernames at or above that limit rather than silently truncating them. The notes describe the intended policy as a maximum of 50 characters, so the constant and message should be made consistent with the actual accepted length.

Issue 15 — Special files at hardcoded paths can block the program

Fix status in notes: NO

Vulnerability: Vulnerability: The program assumes paths such as /vulnerable/log are regular files. If an attacker can replace the log path with a FIFO, a write may block while waiting for a reader.

Exploit / example: Example test described in notes: Remove /vulnerable/log, create a named pipe at that path with mkfifo, then run a command that triggers logging. The process may hang.

Impact: Impact: A blocked process can deny service, affecting availability.

Fix / recommendation: Suggested fix: Ensure the expected paths are regular files and not symlinks/FIFOs or other special files, using safe open flags and fstat() checks. Avoid opening attacker-controlled paths, and handle failures without blocking.

Summary of issue status in the supplied notes

Marked YES: Issues 1–10 and 14 (11 issues).

Marked NO: Issues 11, 12, 13, and 15 (4 issues).

This status summary reflects the text supplied in chat; it does not independently verify the source code or the final submission.

Build command

cc account.c -o account -lcrypto